

44. Le panneau, et les nuances

Un score qui reste en place pendant que le décor glisse.

1. Ce qu'il faut comprendre

Le décor défile ; le score, lui, ne doit pas bouger. Sans une seconde couche, il faudrait le redessiner à chaque image à la position que le défilement lui a donnée — et le moindre décalage se verrait.

Le **panneau** est cette couche : le matériel l'appelle la fenêtre, il la pose par-dessus le décor, et **elle ne défile pas**. `panneau(x, y)` la place, en pixels, et l'allume. On y écrit avec `textePanneau()` et `poserPanneau()`, exactement comme sur le fond.

Il n'y a que **deux cartes de fond** dans la console : le décor prend la première, le panneau la seconde. C'est un choix, et il vaut mieux le savoir que le découvrir.

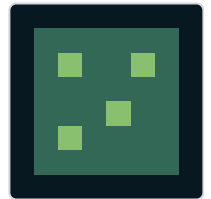
`paletteFond(n0, n1, n2, n3)` choisit les quatre nuances, de la plus claire à la plus sombre — chaque nombre va de 0 à 3. Elles étaient posées une fois pour toutes à l'allumage ; ici on s'en sert pour assombrir le fond par paliers.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1553 octets** de cartouche.

1. UN DESSIN — SOL

```
Tuile SOL = {  
    "33333333",  
    "32222223",  
    "32122123",  
    "32222223",  
    "32221223",  
    "32122223",  
    "32222223",  
    "33333333",  
};
```



à l'écran

2. UNE VARIABLE — D

```
uint8_t d = 0;
```

3. UNE VARIABLE — ATTENTE

```
uint8_t attente = 0;
```

4. UNE VARIABLE — NOIRCEUR

```
uint8_t noirceur = 0;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    for (uint8_t x = 0; x < 32; x++) {
        poser(x, 12, SOL);
        poser(x, 13, SOL);
    }

    textePanneau(1, 0, "SCORE 00");
    panneau(0, 128);

    while (true) {
        image();

        d++;
        defiler(d, 0);

        attente++;

        if (attente > 40) {
            attente = 0;
            noirceur++;
            if (noirceur > 3) noirceur = 0;

            // le fond s'assombrit, puis revient
            paletteFond(noirceur, noirceur + 1, 2, 3);
        }
    }

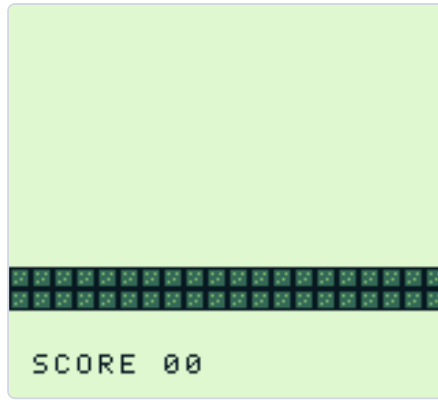
    return 0;
}
```

3. Ce qui se passe, étape par étape

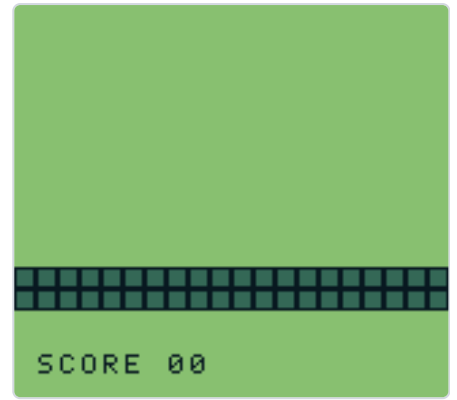
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



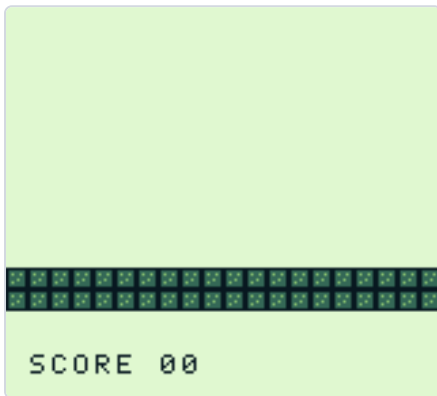
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le sol glisse ; « SCORE 00 » reste collé en bas, et les nuances changent.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le panneau est allumé
- ✓ il lit la seconde carte de fond
- ✓ il est posé à la ligne 128
- ✓ le décor défile
- ✓ mais le panneau ne bouge pas
- ✓ et il n'est pas vide
- ✓ les nuances ont changé (\$e4 vers \$ee)

Ensuite : **45. Remplir un tableau, puis le dessiner** — niveau 6 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++